Investigating the relationship between the angles and the position of the end effector of a path following a robotic arm

Number of Pages: 24

Table of Content

I. Introduction **2**

Overview of Robotic Arms 4

Degrees of Freedom (DOF) 4

Applications of Planar Robotic Arms 4

Kinematics in Robotics 4

II. Mathematical Tools: **5**

arctan2(y, x) Function 5

III. Deriving the Robotic Arm's Kinematics **6**

Deriving Forward Kinematics 7

Deriving Inverse kinematics 9

Deriving Theta2 10

Deriving Theta1 12

Calculated Example 15

Differentiation of Theta1 and Theta2 15

Differentiating Theta2 15

Differentiating Theta1 17

Calculated Example 19

IV. Applications **21**

V. Reflection **21**

Limitations 21

Strengths 21

Improvements 22

VI. Conclusion **22**

VII. References **23**

VIII. Appendix **24**

I. Introduction

Robots govern life around us, from manufacturing our cars to conducting life-saving medical operations. This is what sparked my love for robotics: It's an adaptive field that can be employed in any scenario and makes our lives easier. Robotics consists of machines that perform tasks autonomously or with minimal human intervention. This field combines various disciplines such as engineering, computer science, and mathematics. Planning the robot's motion is an essential aspect, this is why kinematics are a crucial part of robotic design (*Robotic Systems*, n.d.). They allow precise positioning and movement of robotic arms and manipulators, particularly important in repetitive and accuracy-demanding applications such as manufacturing. One night while studying, I looked at my desk lamp, and it got me thinking, how can a lamp that has rotational joints, be able to translate the light on a straight line wherever I want it? This question reminded me of the robotic arms we see in modern factories. This sparked a question in my mind, leading me to investigate what change in angles has to be done to make the tip of a mechanical system follow my desired path. Therefore I came up with this investigation in which a 2-DOF robotic arm will be modeled to follow a described path. This mathematical modeling of the robotic arm involves forward kinematics and inverse kinematics. Solving inverse kinematics is fundamental in trajectory planning, where the robot's end-effector follows a described path over time. This enables it to perform complex tasks such as surgeries, where precision and smooth motion are vital.

This investigation aims to model a two-degree-of-freedom (2-DOF) robotic arm operating in a two-dimensional plane enabling its end-effector to follow a desired trajectory described by function. The investigation derives mathematical equations

that relate the robot's joint angles to the desired position and velocity of the end-effector, enabling the arm to follow a described path smoothly.

Overview of Robotic Arms

Degrees of Freedom (DOF)

Degrees of freedom (DOF) is the number of individual movements a robot or a mechanical system can perform. Each translational or rotational movement along a single axis is a degree of freedom. This combination of movements allows the robot to position and orient itself in space, enabling it to perform various tasks (Craig, 2018). For example, a door has one degree of freedom, a rotational movement along the y-axis. Generally, the complexity of the tasks the mechanical system can perform increases with the more degrees of freedom it has.

Applications of Planar Robotic Arms

A planar robotic arm operates in a two-dimensional XY plane. These robots consist of joints and links allowing rotational or translational motion in two dimensions (Niku, 2020). A planar robotic arm is simpler to model and control relative to three-dimensional robotic arms since their kinematics involve fewer variables. Planar robotic arms can be employed in industrial automation such as CNC machines, plotters, and laser cutters where precision movements in a plane are required (Siciliano & Khatib, 2016).

Kinematics in Robotics

In robotics, kinematics defines the relationships between the robot’s joint parameters and the position and orientation of the end-effector (Craig, 2018).

Forward Kinematics uses the joint parameters (angles or displacements) to determine the position and orientation of the robot's end-effector. It answers the question: knowing the joint variables, where is the end-effector? (Biomechatronics, 2019). Inverse Kinematics are used to calculate the joint parameters needed to reach the desired position and orientation of the end-effector. It answers the question: To get to the desired end-effector position, what should the joint variables be? (Biomechatronics, 2019). Inverse kinematics can vary in complexity due to the possibility of multiple solutions and the nonlinear equations involved.

II. Mathematical tools:

arctan2(y, x) Function

The Arc tangent of two numbers returns the angle θ in radians between the positive x-axis and the vector to the point (x, y). It is similar to calculating the arc tangent of y/x, except that the signs of both arguments are used to determine the quadrant of the result expressed in radians. This function has a range of angles $-\pi \; to \; \pi$.

$$\operatorname{atan2}(y,x) = \begin{cases} \arctan(\frac{y}{x}) & \text{if } x>0, \\ \arctan(\frac{y}{x}) + \pi & \text{if } x<0 \text{ and } y \geq 0, \\ \arctan(\frac{y}{x}) - \pi & \text{if } x<0 \text{ and } y<0, \\ \frac{\pi}{2} & \text{if } x=0 \text{ and } y>0, \\ -\frac{\pi}{2} & \text{if } x=0 \text{ and } y<0, \\ \text{undefined} & \text{if } x=0 \text{ and } y=0. \end{cases}$$

Arctan2 helps avoid issues related to discontinuity or limited range of the standard arctangent function, which can lead to errors in angle calculations and unexpected behaviour (*ATAN2 Function Calculator and Graph*, n.d.).

Figures:

All the figures used in this investigation were generated using a Python code I wrote.

III. Deriving the Robotic Arm's Kinematics

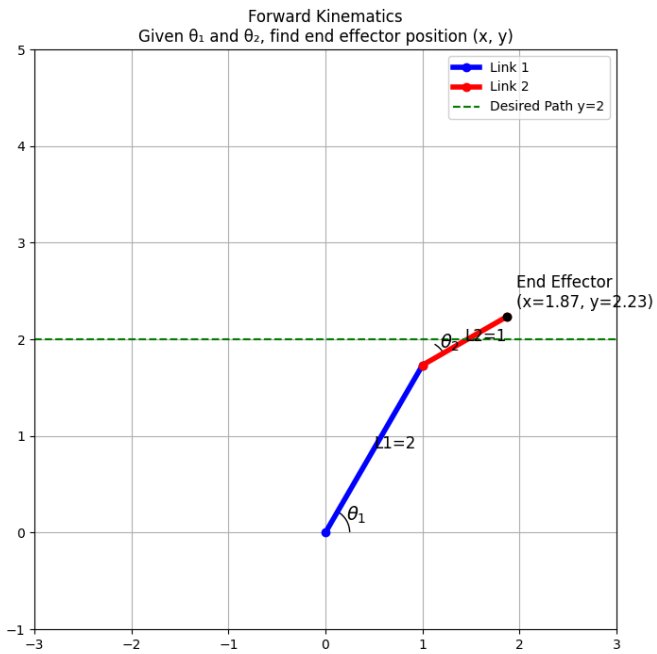


Figure 1: Model of a 2-DOF 2D robotic arm at a position (x, y)

This is an example of my robotic arm, which will be able to move in the 2 quadrants with positive y. In forward kinematics, the angles of the robot are given, and these angles will be used to calculate the position of the tip of the robot known as the end effector.

The desired path from the robot in this case is y=2, this will be just used as an example.

In this diagram, the robot is clearly out of the path, but this is designed just to show the model of the robot, which showcases 2 segments of different lengths, and one articulation in the middle.

Deriving Forward kinematics

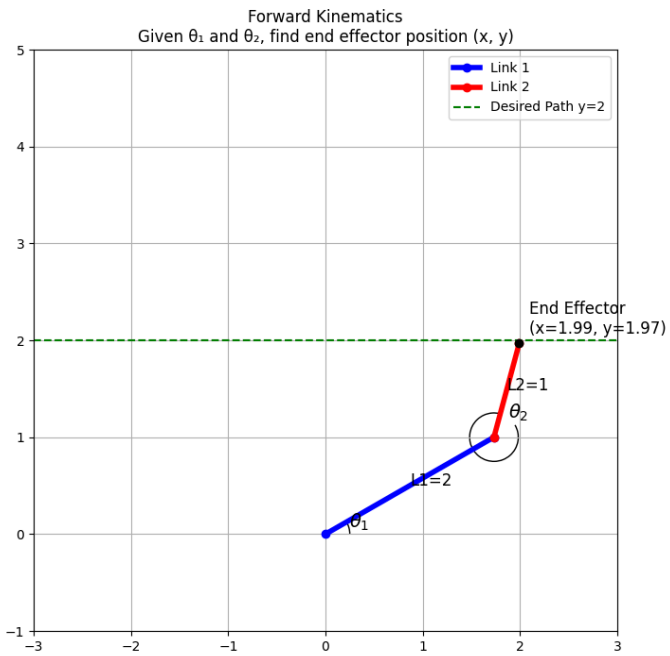


Figure 2: Forward Kinematics of the 2-DOF 2D robotic arm

The position (x_1, y_1) of the end of Link 1 is determined by its length L_1 and angle θ_1:

$$x_1 = L_1 \cos(\theta_1)$$
$$y_1 = L_1 \sin(\theta_1)$$

Calculating the Position of the End Effector

The end effector is at the end of Link 2, which extends from the end of Link 1.

To find its position (x, y), we need to:

1. Determine the angle that Link 2 makes with the horizontal x-axis.

- The total angle from the base to the end of Link 2 is $\theta_1 + \theta_2$.
- θ_1 is the angle from the base to the end of Link 1.
- θ_2: is the angle between Link 1 and Link 2.
- So, the angle of Link 2 with respect to the x-axis is $\theta_1 + \theta_2$.

2. Calculate the horizontal and vertical components of Link 2.
   - Horizontal Component:

     $$\Delta x = L_2 \cos(\theta_1 + \theta_2)$$

   - Vertical Component:

     $$\Delta y = L_2 \sin(\theta_1 + \theta_2)$$

3. Add these components to the position of the first joint.
   - Total x-coordinate:

     $$x = x_1 + \Delta x = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2)$$

   - Total y-coordinate:

     $$y = y_1 + \Delta y = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2)$$

Thus, we obtain the forward kinematics equations:

$$x = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2)$$
$$y = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2)$$

Moving According to the Trajectory

The aim is for the end effector to follow $y = 2$ while x moves from $-2 \; to \; 2$ over time.

Deriving Inverse kinematics

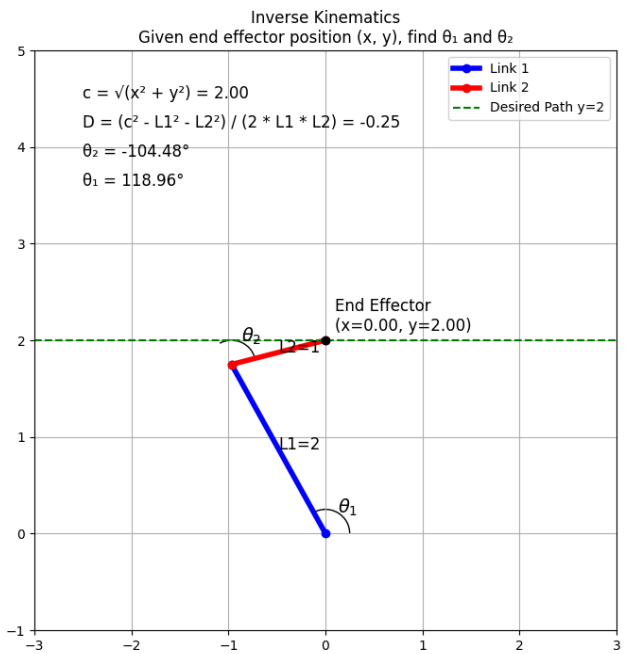


Figure 3: Inverse kinematics of the 2-DOF 2D robotic arm

Inverse Kinematics involves calculating the joint angles θ_1 and θ_2 given the desired end effector position (x, y) .

We have the equations:

$$x = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2)$$
$$y = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2)$$

The goal is to solve for θ_1 and θ_2 given x and y .

When a position of the end effector is desired, therefore we have the x and y coordinates, since the robotic arm is a 2-segmented one, we can consider the triangle formed by the robotic arm and the segment between the base (the joint (0.0)) of the robot and the desired position of the end effector.

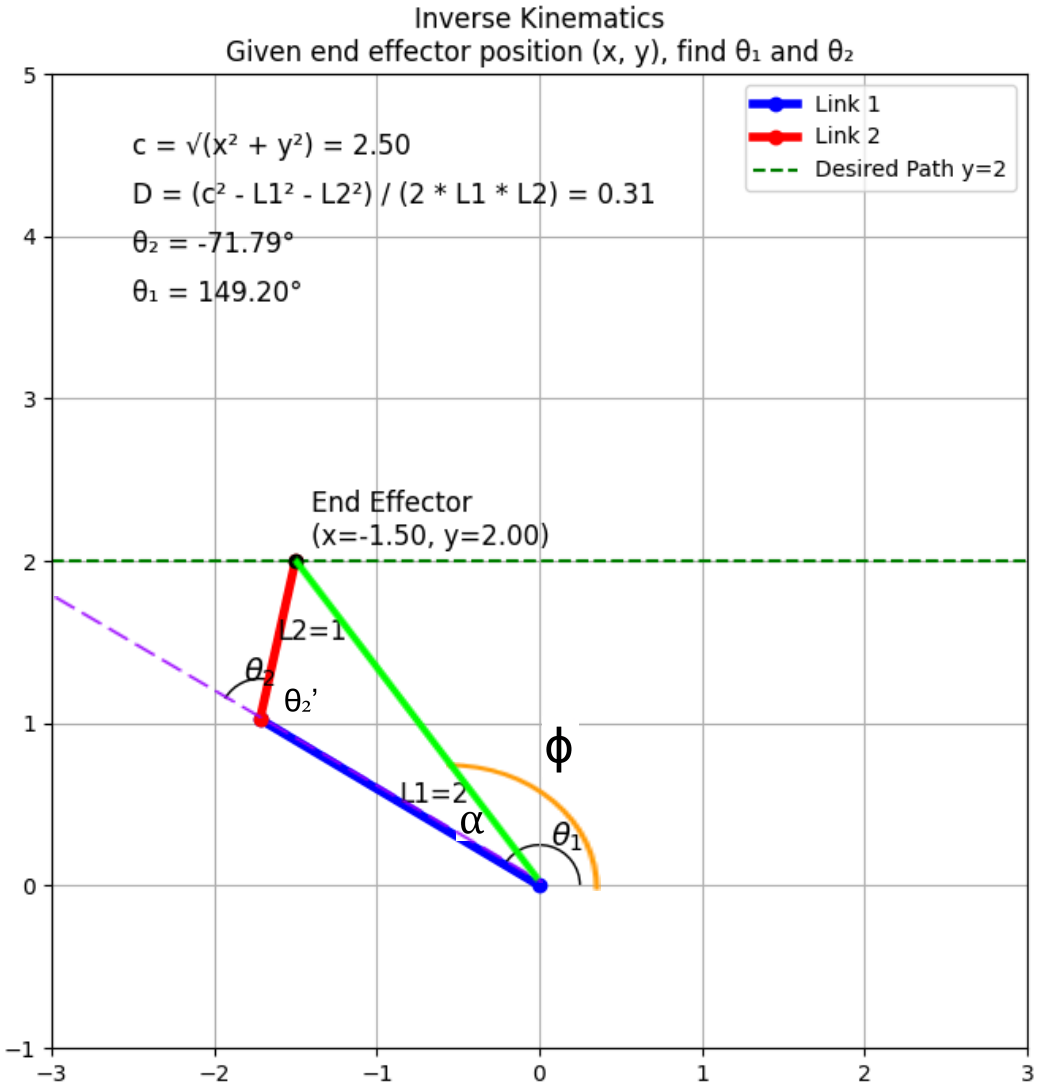


Figure 4: Model demonstrating the calculations for inverse kinematics of the robotic arm

The green line in this figure represents the 3rd segment, based on which we will have the triangle.

First, we calculate the magnitude of this green line noting it as c:

$$c = \sqrt{x^2 + y^2}$$

Deriving θ_2:

Then we can use the Law of Cosines to start solving for θ_2:

$$c^2 = L_1^2 + L_2^2 - 2L_1L_2\cos(\theta_2')$$

Where:

Side L_1 : Length of Link 1.

Side L_2: Length of Link 2.

Side c: Distance from base to end effector.

Angle θ_2': the angle opposite side c, not the same as θ_2. However, we can relate θ_2' and θ_2 using:

- $\theta_2' = \pi - \theta_2$

In the triangle, θ_2' is the internal angle relative to θ_2. Since θ_2 is the external joint angle between the links, θ_2' is supplementary to θ_2. Which can be seen in the figure in the purple discontinued line.

Then we substitute $\theta_2' = \pi - \theta_2$:

$$c^2 = L_1^2 + L_2^2 - 2L_1L_2\cos(\pi - \theta_2)$$

Use the identity $\cos(\pi - \theta) = -\cos(\theta)$:

$$c^2 = L_1^2 + L_2^2 - 2L_1L_2(-\cos(\theta_2))$$
$$c^2 = L_1^2 + L_2^2 + 2L_1L_2\cos(\theta_2)$$

Then we solve for $\cos(\theta_2)$:

Rearrange the equation:

$$c^2 = L_1^2 + L_2^2 + 2L_1L_2\cos(\theta_2)$$

Subtract $L_1^2 + L_2^2$ from both sides:

$$c^2 - L_1^2 - L_2^2 = 2L_1L_2\cos(\theta_2)$$

Divide both sides by L_1L_2:

$$\cos(\theta_2) = \frac{c^2 - L_1^2 - L_2^2}{2L_1L_2}$$

Then we can denote $\cos(\theta_2) = D$:

$$D = \frac{c^2 - L_1^2 - L_2^2}{2L_1L_2}$$

Constraints on D:

The value of D must satisfy $-1 \leq D \leq 1$ for a real solution of θ_2. If $|D| > 1$, the point (x, y) is unreachable by the arm.

Then, after calculating D we can calculate θ_2:

$$\theta_2 = \cos^{-1}(D)$$

There are two possible solutions for θ_2:

- Elbow Up: $\theta_2 = \cos^{-1}(D)$
- Elbow Down: $\theta_2 = -\cos^{-1}(D)$

The elbow up/elbow down configurations refer to whether the arm rotated counterclockwise or clockwise respectively. Or in other words, rotation to the left (+) or rotation to the right (-).

Deriving θ_1:

Now we can calculate the θ_1:

θ_1 is the angle of the rotation of L_1 to the left.

First, we calculate the angle between the x-axis and the line connecting the base to the end effector ϕ:

$$\phi = \operatorname{atan2}\left(\frac{y}{x}\right)$$

Then to determine the angle between L_1 and c noted as α which is calculated by dividing the vertical adjustment of θ_2 by its horizontal adjustment:

$$\tan(\alpha) = \left(\frac{L_2 \sin(\theta_2)}{L_1 + L_2 \cos(\theta_2)} \right)$$

And to calculate it:

$$\alpha = \tan^{-1} \left(\frac{L_2 \sin(\theta_2)}{L_1 + L_2 \cos(\theta_2)} \right)$$

Therefore, to finally get the angle θ_1**:**

$$\theta_1 = \phi \pm \alpha$$

The sign depends on whether θ_2 is calculated with the elbow up or elbow down configuration. For elbow up, we add the two angles, however for elbow down we subtract α from ϕ, to finally get the angle θ_1**.** This is because of the property:

$$\sin(-\theta_2) = -\sin(\theta_2)$$

This affects the sign of α.

Following the trajectory:

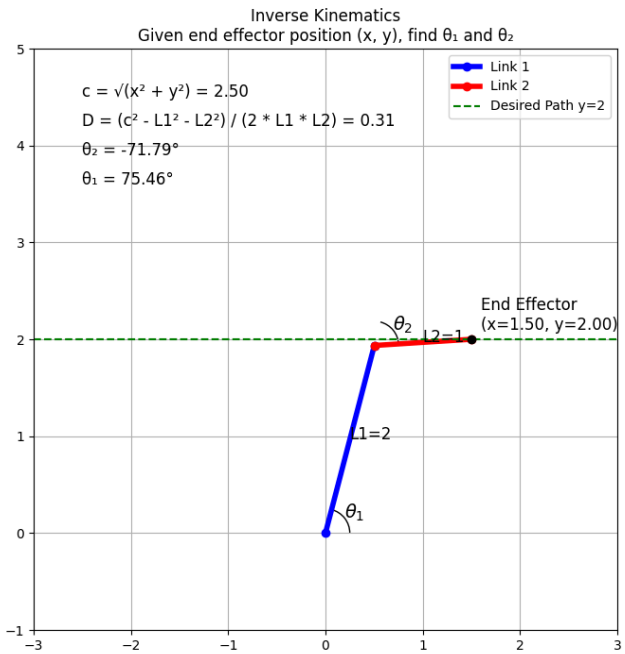


Figure 5: Model showing the use of inverse kinematics to follow the described path

Now, to make the robot follow a certain path with the end effector, we can first model our path using a function. We can start with the simple path as determined earlier:

$$x(t) = -2 + 4 \cdot t$$
$$y(t) = 2$$

Where t is the time at which we want to calculate the angles.

For this case elbow down configuration will be used.

Therefore, we can model our angle θ_2 in terms of (x, y) :

$$\theta_2(x, y) = -\cos^{-1}\left(\frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}\right)$$

Then we can model the angle θ_1 in terms of (x, y) and θ_2:

$$\theta_1(x, y, \theta_2) = \text{atan2}(y, x) - \tan^{-1}\left(\frac{L_2 \sin(\theta_2)}{L_1 + L_2 \cos(\theta_2)}\right)$$

Calculated Example:

Calculating the angles of the robotic arm for:

- $t = 0.5$
- $L_1 = 2,\ L_2 = 1$

The position of the end effector will be:

$$x(0.5) = -2 + 4 \times 0.5 = 0$$

$$y(0.5) = 2$$

Therefore, by plugging the values into the equation of θ_2 we get:

$$\theta_2(0.5) = -\cos^{-1}(-0.25)$$

Giving us:

$$\theta_2(0.5) = -1.82348 \text{ radians}$$

Which is approximately -104°

And for θ_1**:**

$$\theta_1(0.5) = \frac{\pi}{2} - (-0.505361) = \frac{\pi}{2} + 0.505361 \approx 2.07616 \text{ radians}$$

Which is approximately 119°

It is also important to calculate the angular speed of the robotic arm, which describes its linear speed and ensures a smooth and safe robotic performance.

Differentiation of $\theta_2(t)$ and $\theta_1(t)$

The down configuration will be used for these differentiations

Differentiating $\theta_2(t)$:

$$\theta_2(t) = -\cos^{-1}\left(\frac{x(t)^2 + y(t)^2 - L_1^2 - L_2^2}{2L_1L_2}\right)$$

Let:

$$c(t) = \frac{x(t)^2 + y(t)^2 - L_1^2 - L_2^2}{2L_1L_2}$$

First we differentiate $\theta_2(t)$ in terms of t:

$$\dot{\theta}_2(t) = \frac{d\theta_2}{dt} = \frac{d}{dt}\left(-\cos^{-1}(c(t))\right) = \frac{\dot{c}(t)}{\sqrt{1-[c(t)]^2}}$$

Then $\dot{c}(t)$ is derived in terms of t:

To simplify we can denote:

Numerator $N(t)$:

$$N(t) = x(t)^2 + y(t)^2 - L_1^2 - L_2^2$$

Denominator D:

$$D = 2L_1L_2$$

Since L_1 and L_2 are constants, L_2 is also a constant, therefore we can write:

$$\dot{c}(t) = \frac{d}{dt}\left[\frac{N(t)}{D}\right] = \frac{1}{D} \cdot \frac{dN(t)}{dt}$$

So,

$$\dot{c}(t) = \frac{\dot{N}(t)}{D}$$

Then we differentiate $N(t)$ in terms of t:

Since L_1 and L_2 are constants, their squares are constants, and their derivatives are zero.

$$\dot{N}(t) = \frac{d}{dt}[x(t)^2 + y(t)^2 - 0 - 0]$$

We apply the chain rule:

$$\dot{N}(t) = 2x(t)\,\dot{x}(t) + 2y(t)\,\dot{y}(t)$$

Where:

- $\dot{x}(t) = \dfrac{dx(t)}{dt}$
- $\dot{y}(t) = \dfrac{dy(t)}{dt}$

Finally, we substitute $\dot{N}(t)$ Back into $\dot{c}(t)$

We have:

$$\dot{c}(t) = \frac{\dot{N}(t)}{D} = \frac{2x(t)\,\dot{x}(t) + 2y(t)\,\dot{y}(t)}{2L_1L_2}$$

Therefore:

$$\dot{c}(t) = \frac{dc}{dt}\left(\frac{x(t)^2 + y(t)^2 - L_1^2 - L_2^2}{2L_1L_2}\right) = \frac{2x(t)\,\dot{x}(t) + 2y(t)\,\dot{y}(t)}{2L_1L_2} = \frac{x(t)\,\dot{x}(t) + y(t)\,\dot{y}(t)}{L_1L_2}$$

Since we have $\dot{c}(t)$ we substitute it back into $\dot{\theta}_2(t)$ and finally we have:

$$\dot{\theta}_2(t) = \frac{\dot{c}(t)}{\sqrt{1 - [c(t)]^2}} = \frac{x(t)\,\dot{x}(t) + y(t)\,\dot{y}(t)}{L_1L_2\sqrt{1 - [c(t)]^2}}$$

This function depicts the relationship between the rate of change of $\dot{\theta}_2(t)$ and the rate of change of the position of the end effector in terms of t.

Differentiating $\theta_1(t)$:

$$\theta_1(t) = \phi(t) - \tan^{-1}\left(\frac{L_2\sin(\theta_2(t))}{L_1 + L_2\cos(\theta_2(t))}\right)$$

Where:

$$\phi(t) = \operatorname{atan}\left(\frac{y(t)}{x(t)}\right)$$

First, we write $\theta_1(t)$ explicitly which gives us the following:

$$\theta_1(t) = \phi(t) - \tan^{-1}(k(t))$$

Where:

$$k(t) = \frac{L_2 \sin(\theta_2(t))}{L_1 + L_2 \cos(\theta_2(t))}$$

We differentiate $\theta_1(t)$ in terms of $k(t)$ and $\dot{\phi}(t)$:

$$\dot{\theta}_1(t) = \dot{\phi}(t) - \frac{\dot{k}(t)}{1 + [k(t)]^2}$$

Now we workout $\dot{\phi}(t)$:

$$\phi(t) = \operatorname{atan2}\left(\frac{y(t)}{x(t)}\right)$$

Using $\frac{d}{dt}[\tan^{-1}(u)] = \frac{1}{1+u^2} \cdot \frac{du}{dt}$:

We calculate $1 + u^2$:

$$1 + \left(\frac{y(t)}{x(t)}\right)^2 = \frac{x(t)^2 + y(t)^2}{x(t)^2}$$

Applying the rule we get:

$$\dot{\phi}(t) = \frac{1}{1 + \left(\frac{y(t)}{x(t)}\right)^2} \cdot \frac{\dot{y}(t)\,x(t) - y(t)\,\dot{x}(t)}{x(t)^2}$$

Which simplifies to:

$$\dot{\phi}(t) = \frac{x(t)\,\dot{y}(t) - y(t)\,\dot{x}(t)}{x(t)^2 + y(t)^2}$$

Now we differentiate $k(t)$:

$$\dot{k}(t) = \frac{d}{dt}\left(\frac{L_2 \sin(\theta_2(t))}{L_1 + L_2 \cos(\theta_2(t))}\right)$$

Using the quotient rule and chain rule:

Let:

- Numerator: $N(t) = L_2 \sin(\theta_2(t))$
- Denominator: $D(t) = L_1 + L_2 \cos(\theta_2(t))$

Therefore we have:

$$k(t) = \frac{N(t)}{D(t)}$$

Deriving both $N(t)$ and $D(t)$:

- $\dot{N}(t) = L_2 \cos(\theta_2(t))\,\dot{\theta}_2(t)$
- $\dot{D}(t) = -L_2 \sin(\theta_2(t))\,\dot{\theta}_2(t)$

We apply the quotient rule and we get the following:

$$\dot{k}(t) = \frac{\dot{N}(t)\,D(t) - N(t)\,\dot{D}(t)}{[D(t)]^2}$$

Calculated Example:

Calculating the angles of the robotic arm for:

- $t = 0.5$
- $L_1 = 2,\ L_2 = 1$

The velocity of the end effector will be:

- $\dot{x}(t) = \frac{d}{dt}[-2+4t] = 4$
- $\dot{y}(t) = 0$ (since y(t) is a constant function)

So calculating the rate of change of θ_2 which is the angular velocity we get:

$$\dot{c}(0.5) = \frac{0\times 4 + 2\times 0}{2\times 1} = \frac{0}{2} = 0$$

Therefore:

$$\dot{\theta}_2(0.5) = \frac{0}{\sqrt{1-(-0.5)^2}} = \frac{0}{\sqrt{1-0.25}} = \frac{0}{\sqrt{0.75}} = 0 \text{ radians per second}$$

Now we calculate the angular velocity for θ_1:

$$\dot{N}(0.5) = L_2\cos(\theta_2(0.5))\,\dot{\theta}_2(0.5) = 1\times(-0.5)\times 0 = 0$$

$$\dot{D}(0.5) = -L_2\sin(\theta_2(0.5))\,\dot{\theta}_2(0.5) = -1\times(-0.8660)\times 0 = 0$$

$$\dot{k}(0.5) = \frac{0\times 1-(-0.9682)\times 0}{1.75^2} = \frac{0}{3.0625} = 0$$

$$\dot{\phi}(0.5) = \frac{0\times 0 - 2\times 4}{0^2+2^2} = \frac{-8}{4} = -2$$

Therefore:

$$\dot{\theta}_1(0.5) = -2-0 = -2 \text{ radians per second}$$

Which means:

- Joint 2 is stationary at this instant ($\dot{\theta}_2(0.5) = 0$).
- Joint 1 is moving at a rate of -2 radians per second to maintain the end-effector's trajectory.

Limitations

First, this investigation was conducted on a 2-DOF 2D robotic model which although some applications of it exist, is not commonly used in most scenarios. Second, in this investigation, zero dynamics were assumed, therefore it did not take into account parameters such as forces, torques, inertia, and friction which can significantly affect the robot's motion in a real-life scenario.

Strengths

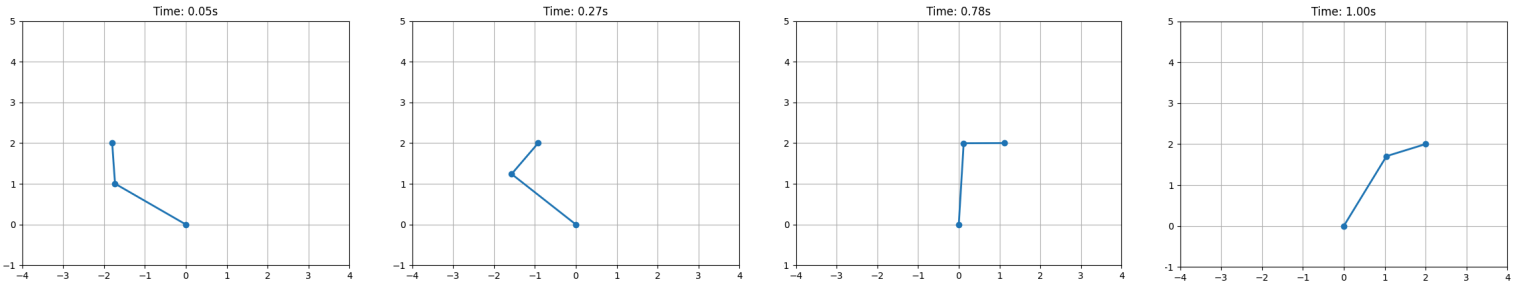
This investigation bridges the gap between mathematics and its applications in engineering such as robotic designing. This investigation deepened my understanding of robotic motion through the step-by-step derivations of the equations and seeing how parameters depend on each other with every step. I also programmed a simulation of a path-following robotic arm using the mathematics I derived, which can be used for education, demonstrations, and prototyping.

Improvements

The scope of this investigation can be expanded to study the kinematics of higher degrees of freedom robotic arm as well as the effect of other external forces and parameters such as temperature and friction in a practical scenario on the path of the end-effector, possibly looking into adaptable solutions that could adapt the robot to different conditions ensuring optimal performance.

IV. Applications

These kinematics equations are essential when designing and building robotic arms that require precision and smooth movements such as surgical operations and manufacturing robots. These equations define and predict the behavior of the robotic arm, allowing for simulation and testing. This ensures an accurate and fast solution since they are not computationally intensive.



These are snapshots from an animated simulation I created with the equations demonstrated in this investigation. The robotic arm in this simulation follows the straight line y=2 with its end-effector.

V. Conclusion

To conclude, this investigation has clearly shown and demonstrated the relationship between the angles of the 2-DOF 2D robotic arm and the path followed by the end-effector. It demonstrates how the second angle depends on the position of the end effector in the path and how the base angle depends on both the position (x,y) of the end effector and the second angle. This relationship has also been demonstrated while deriving the equations for the rate of change and how the angular speed of the angles depends on the speed of the path followed, which in this case was a straight line. This shows how the angular velocity of the angles depends on the linear speed

of the end-effector along the path followed, which was defined by a function. The relationships derived in this investigation were also generalized and applied to any kind of path described. When the point of the path is out of reach, the equations will give an impossible solution, therefore the path's equation needs to be always limited to within the reach of the robotic arm which has been explained in this investigation.

This investigation helped me deepen my knowledge and understanding in the field of robotics which I'm passionate about, allowing me to explore beyond the simple electronics and programming and dig deeper into the mathematics that make the magic of these machines possible.

VI. References

Craig, J. J. (2018). "Introduction to robotics: Mechanics and control" (4th ed.). Pearson Education.

Lynch, K. M., & Park, F. C. (2017). "Modern robotics: Mechanics, planning, and control". Cambridge University Press. Retrieved from

http://hades.mech.northwestern.edu/index.php/Modern_Robotics

Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2010). "Robotics: Modelling, planning and control". Springer.

Niku, S. B. (2020). "Introduction to robotics: Analysis, control, applications" (3rd ed.). John Wiley & Sons.

Biomechatronics. (2019). Robot kinematics: Forward and inverse kinematics. Retrieved from

https://biomechatronics.ca/robot-kinematics/

Taylor, R. H., Menciassi, A., Fichtinger, G., Fiorini, P., & Dario, P. (2016). Medical robotics and computer-integrated surgery. In B. Siciliano & O. Khatib (Eds.), "Springer handbook of robotics" (pp. 1657–1684). Springer.

ATAN2 function calculator and graph. (n.d.). MedCalc. Retrieved October 6, 2024, from https://www.medcalc.org/manual/atan2-function.php

Robotic Systems. (n.d.). Kinematics. Retrieved January 14, 2025, from https://motion.cs.illinois.edu/RoboticSystems/Kinematics.html

VII. Appendix

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
# Define the robotic arm parameters
L1 = 2  # Length of the first link
L2 = 1  # Length of the second link
T = 1.0         # Total time duration
dt = 0.01      # Time step
time_steps = np.arange(0, T + dt, dt)
def desired_trajectory(t):
    x = -2 + 4 * t / T  # x moves from -2 to 2 over time T
    y = 2               # y is constant at 2 units
    return x, y
def inverse_kinematics(x, y, L1, L2):
    c = np.hypot(x, y)
    D = (x**2 + y**2 - L1**2 - L2**2) / (2 * L1 * L2)
    if np.abs(D) > 1.0:
        raise ValueError("The point is unreachable.")
    theta2 = np.arctan2(-np.sqrt(1 - D**2), D)
    phi = np.arctan2(y, x)
    psi = np.arctan2(L2 * np.sin(theta2), L1 + L2 * np.cos(theta2))
    theta1 = phi - psi
    return theta1, theta2
theta1_list = []
theta2_list = []
x1_list = []
y1_list = []
x2_list = []
y2_list = []
for t in time_steps:
    x, y = desired_trajectory(t)
    try:
        theta1, theta2 = inverse_kinematics(x, y, L1, L2)
    except ValueError as e:
```

```
        print(f"At time {t:.2f}s: {e}")
        continue
    x1 = L1 * np.cos(theta1)
    y1 = L1 * np.sin(theta1)
    x2 = x1 + L2 * np.cos(theta1 + theta2)
    y2 = y1 + L2 * np.sin(theta1 + theta2)
    theta1_list.append(theta1)
    theta2_list.append(theta2)
    x1_list.append(x1)
    y1_list.append(y1)
    x2_list.append(x2)
    y2_list.append(y2)
fig, ax = plt.subplots()
ax.set_xlim(-4, 4)
ax.set_ylim(-1, 5)
ax.set_aspect('equal')
ax.grid(True)
line, = ax.plot([], [], 'o-', lw=2)
def init():
    line.set_data([], [])
    return line,
def update(frame):
    x_data = [0, x1_list[frame], x2_list[frame]]
    y_data = [0, y1_list[frame], y2_list[frame]]
    line.set_data(x_data, y_data)
    ax.set_title(f"Time: {time_steps[frame]:.2f}s")
    return line,
ani = FuncAnimation(fig, update, frames=len(time_steps),
                    init_func=init, blit=True, interval=50)
plt.show()
```